

COURSE: PHYSICS-II
PROJECT PRESENTATION ON:
Cardio Care: Real Time Heart Rate Monitoring



Instructor:
Dr. N. Rakesh Chandra
Head of Department of Physics

Presented by:
Varshini
(AUU23EGCSE065)



Abstract:

This project aims to demonstrate the monitoring of heart rate using a pulse sensor interfaced with Arduino. The project includes an overview of the pulse sensor, its connection with Arduino, and various applications such as LED blinking with heartbeat synchronization, plotting heartbeat signal, and visualizing pulse sensor data in real-time using Processing software.



Research Question:

Main Question: How accurately can the SEN-11574 pulse sensor monitor heart rate when interfaced with an Arduino?

Hypothesis:

The SEN-11574 pulse sensor, when correctly interfaced with an Arduino, can accurately measure heart rate by detecting light absorption variations due to blood flow. This data can be processed and visualized in real-time applications, providing valuable insights for health monitoring. The project aims to demonstrate this functionality and explore its practical applications.

Components:

Arduino UNO Board:

Specifications: Depends on the specific Arduino model (e.g., Arduino Uno, Nano, Mega), Typically operates at 5V.

Purpose: Microcontroller platform for building interactive electronic projects., Used for prototyping, automation, and IoT applications.



Pulse sensor (SEN-11574):

Specifications: Operating Temperature: -40°C to $+80^{\circ}\text{C}$, Input Voltage (VCC): 3V to 5.5V, Output Voltage: 0.3V to VCC, Supply Current: 3mA to 4mA, Dimensions (L x W): 15.8mm

Purpose: Measures heart rate by detecting changes in blood volume., Used for fitness monitoring, healthcare applications, and research.



Jumper Wires:

Specifications: Varied lengths and gauges available., Male-to-male, male-to-female, and female-to-female connectors.

Purpose: Establish electrical connections between components in circuits., Facilitate data and power transmission in electronic projects.



Processing software:

Specifications: Compatible with various operating systems (Windows, macOS, Linux)., Open-source programming language and integrated development environment (IDE).

Purpose: Enables visualization, animation, and interaction in electronic projects., Used for data processing, simulation, and creating graphical user interfaces (GUIs).



Experimental Design:

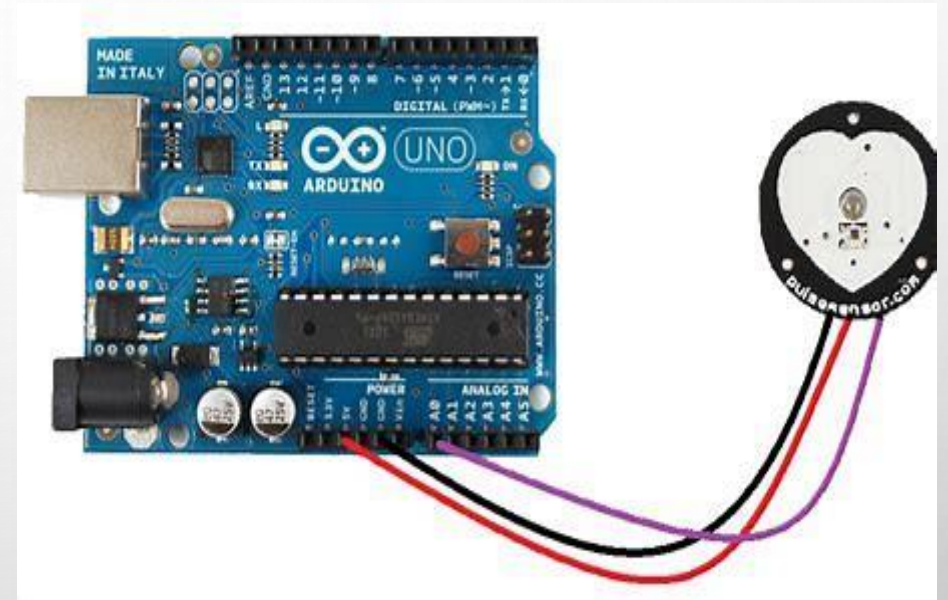
1. Introduction to Pulse Sensor (SEN-11574):

- SEN-11574 pulse sensor: Compact and user-friendly device for heart rate monitoring.
- Detects changes in light absorption caused by blood flow in capillary tissues.
- Provides real-time heart rate data.
- Easy interfacing with microcontrollers like Arduino via three pins (GND, VCC, A0).
- Suitable for various applications including health monitoring and fitness tracking.
- Reliable, accurate, and low power consumption.
- Valuable tool for hobbyists, developers, and researchers interested in physiological monitoring and analysis.



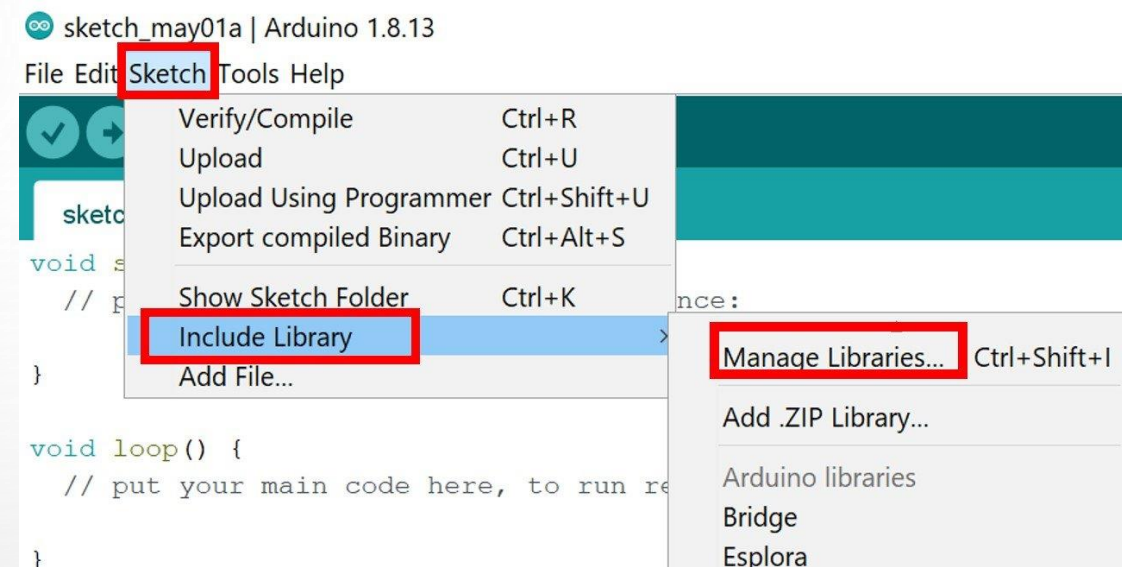
2. Interfacing Pulse Sensor with Arduino:

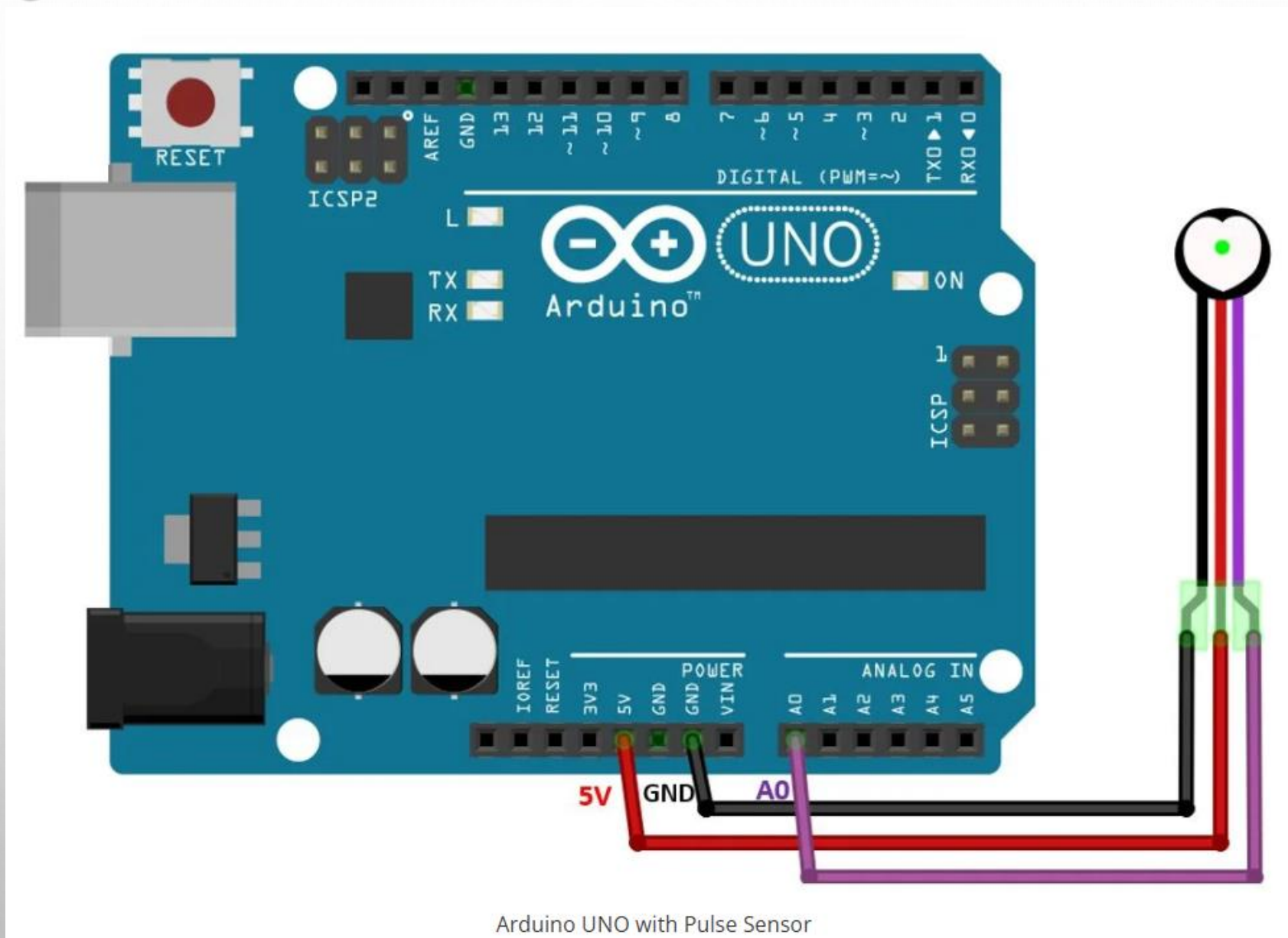
- Connect GND pin of pulse sensor to GND pin of Arduino.
- Connect VCC pin of pulse sensor to 5V pin of Arduino.
- Connect signal (A0) pin of pulse sensor to any analog input pin of Arduino (e.g., A0).
- Install Pulse Sensor Arduino Library through Arduino IDE.
- Use **analogRead()** function to read sensor data from the analog pin.
- Implement example sketches provided in the library for different applications.
- Ensure proper handling of sensor data to monitor heart rate accurately.



3. Installation of Pulse Sensor Arduino Library:

- Open Arduino IDE.
- Navigate to Sketch > Include Library > Manage Libraries.
- In the Library Manager, search for "pulsesensor".
- Select the PulseSensor Playground library from the search results.
- Click the "Install" button to install the library.
- Once installed, restart the Arduino IDE to apply changes.
- The PulseSensor Arduino Library is now ready for use in your projects.





Arduino UNO with Pulse Sensor

4. Implementation of Example Sketches for Arduino:

arduino

```
int pulsePin = A0;    // Pulse sensor analog pin
int ledPin = 13;      // On-board LED pin
int threshold = 550;  // Pulse sensor threshold

void setup() {
    pinMode(ledPin, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    int pulseValue = analogRead(pulsePin);

    if (pulseValue > threshold) {
        digitalWrite(ledPin, HIGH);
    } else {
        digitalWrite(ledPin, LOW);
    }

    delay(10);
}
```

LED Blinking with heartbeat:

- Set up Arduino and pulse sensor connections.
- Define variables for pulse sensor pin and LED pin.
- Configure pulse sensor threshold.
- Read pulse sensor signal using `analogRead()`.
- Toggle LED based on signal threshold.

Plotting Heartbeat Signal:

- Similar setup as LED blinking.
- Read pulse sensor signal and print to Serial Monitor.
- Open Serial Plotter in Arduino IDE.
- View real-time plot of heartbeat signal.

arduino

```
int pulsePin = A0;    // Pulse sensor analog pin

void setup() {
  Serial.begin(9600);
}

void loop() {
  int pulseValue = analogRead(pulsePin);
  Serial.println(pulseValue);
  delay(10);
}
```


Monitoring BPM with Pulse Sensor:

- Initialize PulseSensorPlayground library.
- Configure pulse sensor settings including threshold.
- Read BPM (Beats Per Minute) using `getBeatsPerMinute()`.
- Print BPM to Serial Monitor.
- Detect heartbeat and print message when detected.

```
#include <PulseSensorPlayground.h>

const int pulsePin = A0; // Pulse sensor analog pin
const int ledPin = 13;   // On-board LED pin
const int threshold = 550; // Pulse sensor threshold

PulseSensorPlayground pulseSensor;

void setup() {
  Serial.begin(9600);
  pulseSensor.analogInput(pulsePin);
  pulseSensor.blinkOnPulse(ledPin);
  pulseSensor.setThreshold(threshold);

  if (!pulseSensor.begin()) {
    Serial.println("Error initializing pulse sensor!");
    while (1);
  }
}

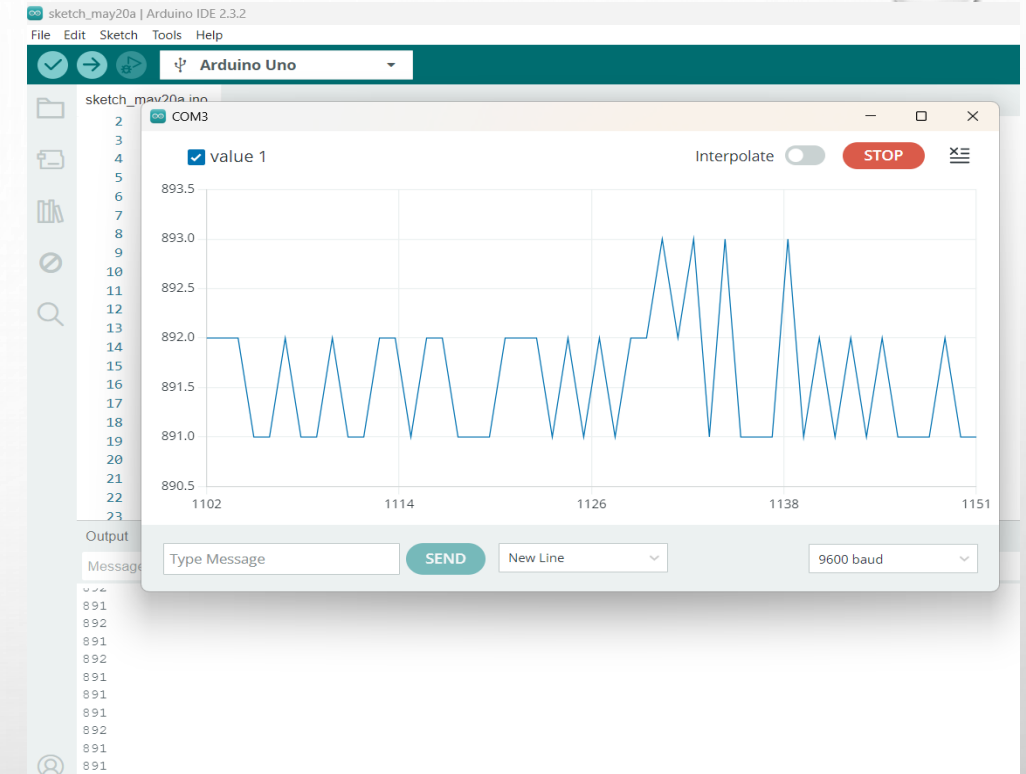
void loop() {
  int bpm = pulseSensor.getBeatsPerMinute();
  if (pulseSensor.sawStartOfBeat()) {
    Serial.println("♥ A HeartBeat Happened !");
    Serial.print("BPM: ");
    Serial.println(bpm);
  }

  delay(20);
}
```

5. Processing Visualizing Pulse Sensor Data:

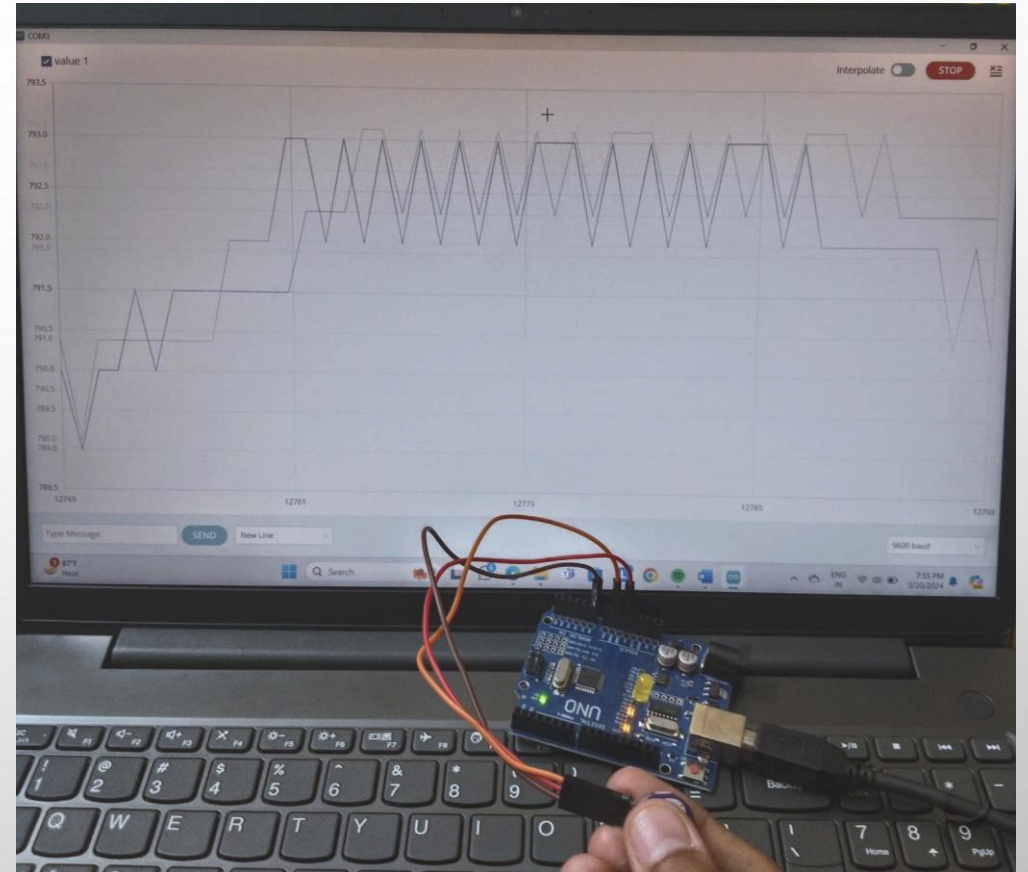
- Open Arduino IDE and upload the "PulseSensor_BPM" example sketch to your Arduino board.
- Connect the pulse sensor to your Arduino board as per the schematic provided.
- Set the OUTPUT_TYPE variable in the Arduino sketch to PROCESSING_VISUALIZER.
- Install Processing software on your computer if not already installed.
- Download the PulseSensorAmpd_Processing_Visualizer code from GitHub.
- Extract the downloaded zip file and copy the folder to the Processing Sketchbook directory.
- Open Processing software and navigate to Sketchbook > PulseSensorAmpd_Processing_Visualizer.
- Select the correct serial port for your Arduino board in Processing.
- Run the Processing sketch to visualize the pulse sensor data in real-time, including BPM and IBI.

Live demonstration link: [**DEMONSTRATION VIDEO LINK.mp4**](#)



Result:

- Interfacing pulse sensor with Arduino: Successful establishment of communication between the pulse sensor (SEN-11574) and Arduino board.
- Accurate heart rate monitoring: Demonstrated through LED blinking synchronized with heartbeat and real-time plotting of heartbeat signal using Serial Plotter.
- Real-time visualization: Utilized Processing software to visualize pulse sensor data, including BPM (Beats Per Minute) and IBI (Inter-Beat Interval), in real-time.



Applications of Real-Time Heart Rate Monitoring:

- Healthcare Monitoring: Patient monitoring in hospitals or home care, Elderly care with alerts for caregivers.
- Fitness and Sports: Athlete training optimization, Integration into fitness trackers and wearables.
- Personal Health: Daily health tracking, Stress management through heart rate variability monitoring.
- Research and Education: Biomedical research data collection, Educational tools for learning bio-signals, electronics, and programming.
- Emergency Response: Providing critical data for EMS, Integration with Automated External Defibrillators (AEDs).
- Remote Health Monitoring: Telemedicine with remote patient monitoring, Health care in rural areas with limited access to facilities.



Conclusion:

The project successfully demonstrated the effective monitoring of heart rate using a pulse sensor and Arduino platform. The findings highlight the potential applications of this technology in healthcare, fitness monitoring, and research. However, further optimization and validation are required for medical-grade applications.



